ODE-FFN: Progress Report 1

Independent Study — Deep Learning

Zhizhe Zhang
NYU Shanghai

20 October 2026

1 Project overview

We study whether an ODE-based FFN can improve a flow-matching model, with the longer-term aim of replacing the FFN in JiT. The original block approximates a linear flow using a learned matrix and an input-dependent duration. We use unconditional 2D targets to examine this construction before returning to images.

So far, increasing the accuracy of the internal ODE approximation has not reliably improved generation. Low-order checkpoints perform worse when their polynomial layers are replaced by matrix exponentials. Constraining the operator also lowers measured sensitivity without consistently improving samples.

2 The original ODELayer

For hidden input h, the layer computes

$$F_K(h) = T_K(s(h)P)h, \qquad T_K(A) = \sum_{k=0}^{K} \frac{A^k}{k!}, \qquad s(h) = 0.2 + 0.8\ \mathrm{sigmoid}(g(h)/10), \tag{1}$$

where P is learned and g is a two-layer SiLU network. The same P and duration are reused across all terms. ODEFFN contains two such layers with SiLU between them. At $K = 1$, the layer is simply $h + s(h)Ph$.

For fixed s, $\exp(sP)h$ is the solution of $dy/du = Py$ after duration s. This hidden duration differs from the external generation time t. The exponential series converges for any finite P. With $r = s\|P\|_2$, its remainder is bounded by $e^r r^{K+1}/(K+1)!$. The input gate also contributes to sensitivity:

$$J_{F_K} = T_K(sP) + [P\, T_{K-1}(sP)h]\, \nabla_h s^\top. \tag{2}$$

A constraint on P therefore controls only part of the layer's Jacobian.

3 Setup

The targets are an anisotropic Gaussian with mean $(1, -1)$ and covariance $\mathrm{diag}(0.09, 4)$, eight Gaussian modes on a radius-3 ring with within-mode covariance $0.04I$, and centered two moons with noise standard deviation 0.08. Each has 100,000 training points and 10,000 points each for validation and testing.

We train velocity networks on $x_t = (1-t)x_0 + tx_1$ with target $x_1 - x_0$, standard Gaussian x_0, uniform t, and no labels. All runs use seed 0, width 64, three residual blocks, batch 512, AdamW at 3×10^{-4}, and 5,000 updates unless stated otherwise.

Generation is evaluated by sliced Wasserstein distance SW_2, using 2,048 fixed source/target points and 128 fixed directions. Lower is better. Sampling uses 32 Heun steps, with finer resolutions checked on Ring8. Gaussian and Ring8 also have analytical velocity targets for measuring field error.

4 Baseline comparison

We compare ODE $K = 2$ with SiLU and SwiGLU FFNs at approximately 82,000 parameters. ODE uses 105,152 forward linear MACs per example versus 80,000 for SwiGLU. Training SwiGLU for 6,572 updates approximately matches ODE's cumulative MAC budget at 5,000 updates.

Model / updates	Gaussian	Ring8	Moons
SiLU / 5,000	0.0868	0.1631	0.0448
SwiGLU / 5,000	0.0746	0.2080	0.0302
ODE K2 / 5,000	0.0674	0.1848	0.0271
SwiGLU / 6,572	0.0658	0.1508	0.0624

Table 1: Fixed-endpoint SW_2. The final row matches the ODE $K = 2$ forward-linear-MAC budget; backward, activation and optimizer costs are excluded.

ODE beats same-update SwiGLU on all three targets, while SiLU is better on Ring8. After matching the MAC budget, ODE remains better only on Moons. These comparisons use fixed endpoints; Ring8 SiLU changes from 0.1631 to 0.2258 between updates 5,000 and 5,024, so the ranking is also sensitive to the training endpoint.

5 Does a more accurate ODE solve help?

To separate approximation error from training effects, we freeze checkpoints trained at $K = 1, 2, 4, 8$ and change only the evaluation order, including direct matrix exponentiation. Gate parameters are frozen, but gate values can change with the resulting hidden states. Polynomial and exponential actions use float64; the outer network remains float32. Same-order checks reproduce the native scores.

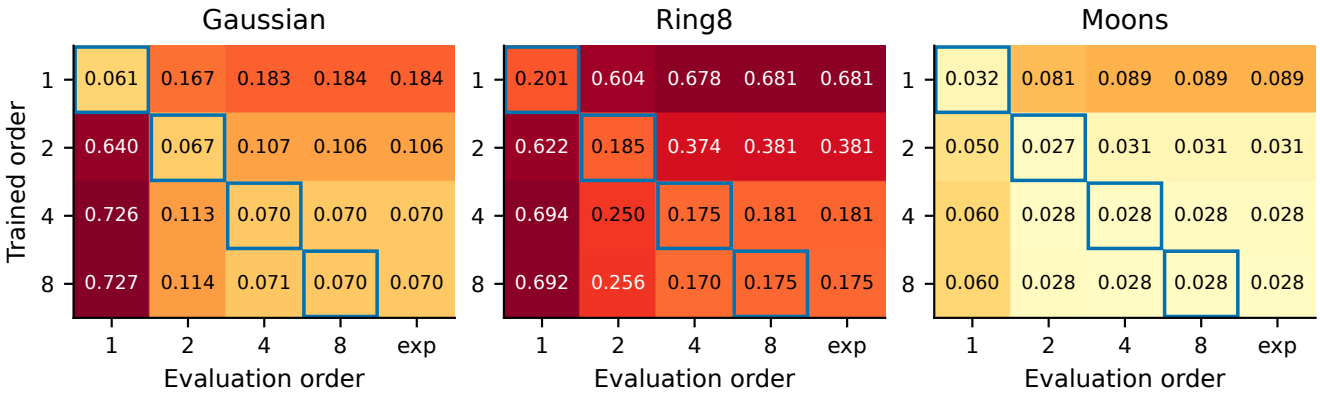


Figure 1: Frozen-checkpoint SW_2. Rows give trained order, columns give evaluation order; blue outlines mark the trained order. All checkpoints are from update 5,000.

Replacing K1/K2 with exponentials worsens generation on every target. For Ring8 K2, SW_2 rises from 0.18482 to 0.38133, and oracle velocity MSE rises from 0.02871 to 0.16944. The trained network has adapted to its polynomial. This does not predict how a model trained with exponentials would perform.

Separately trained Ring8 K4/K8 scores differ by only 0.198%. Yet substituting K8 into the K4 checkpoint worsens SW_2 by 3.225%; the reverse substitution improves it by 2.672%. K8 itself is close to exponential evaluation, changing SW_2 by at most 0.0030% across targets. At 256 Heun steps, Ring8 K2-to-exponential still changes SW_2 from 0.18613 to 0.38371, and K4-to-K8 still worsens it by 3.1%. Finer sampling does not remove these effects.

6 What does P learn, and should we constrain it?

We first cap the spectral norm using $P_{\text{eff}} = P \min(1, 1/\|P\|_2)$. Across K1/K2 and the three targets, the cap lowers the measured layer Jacobian norm in every comparison, but improves SW_2 in only half. On Ring8 K2, the Jacobian norm falls from 3.014 to 2.099, while SW_2 worsens from 0.1848 to 0.1889. These are averages over sampled inputs, not global bounds.

Replacing the input gate by a time-conditioned gate removes the second term in Eq. (2) at fixed t. After retraining, sensitivity does not always fall: Moons K2's Jacobian norm rises from 2.334 to 2.509, while SW_2 improves slightly from 0.02708 to 0.02663. Changes in the learned P and gate values can offset the removed derivative term.

During training, the mean largest singular value of P grows from about 1.109 to 1.715–1.862, while stable rank $\|P\|_F^2/\|P\|_2^2$ falls from 17.37 to 8.31–9.40. The leading direction takes a larger share of the spectral energy. Flattening the singular values could remove useful structure, so its benefit needs to be tested.

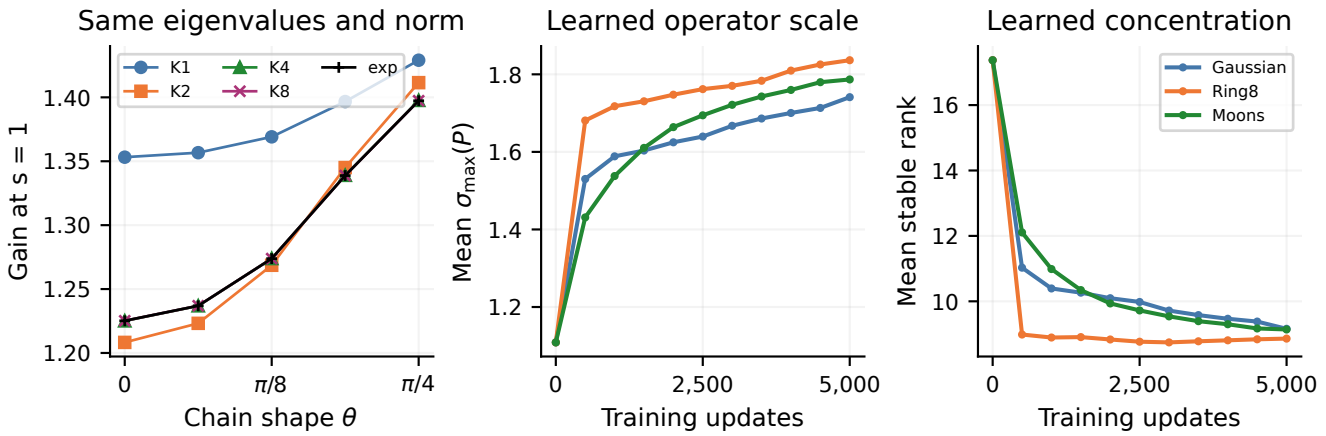


Figure 2: Left: constructed operators with eigenvalues -0.25 and spectral norm 1 still have different finite-time gains. Middle/right: six-layer means of operator scale and stable rank during K2 training.

The matrix example in Fig. 2 uses $P_\theta = -I/4 + a_\theta N_\theta$, where $(N_\theta)_{12} = \cos\theta$, $(N_\theta)_{23} = \sin\theta$, and other entries are zero. Choosing a_θ to keep $\|P_\theta\|_2 = 1$ fixes all eigenvalues at $-1/4$, yet $\|\exp(P_\theta)\|_2$ ranges from 1.225 to 1.397. Eigenvalues and the largest singular value alone do not determine finite-time amplification. This example does not establish which constraint would help training.

The networks have no attention and predict velocity; benefits for JiT image generation remain untested.

7 Hypotheses for the next two weeks

1. **Input-dependent duration improves on a scalar scale.** Compare the original K2 gate with fixed $s = 0.6$ and one learned scalar per layer, retaining both baselines. Choose the endpoint rule on validation data before training. Matching results would weaken the case for an input-dependent gate.
2. **The gate derivative explains sensitivity beyond $\|P\|_2$.** Group held-out inputs by gate-correction norm and measure perturbation amplification through the full block. Use the result to choose a single operator or gate constraint for the next training comparison.